

Tiny Bank: OOP in TypeScript

In-Class Practice · Estimated time: 30–40 minutes

Work in the oop-bank-starter project. Start your repo at <https://classroom50.org/www-csci-345/csci-345-fall-2026/assignments/tiny-bank/accept>.

You may edit any file under `src/` except `bankApp.ts` (the UI) and `main.ts`.

1. The scenario

“Tiny Bank” is a small browser app that manages two accounts, a **savings** account and a **checking** account. You can deposit, withdraw, apply interest, change the interest rate, and transfer money between accounts.

The app **runs**, but it does not behave the way the bank’s rules require, and one feature is missing. A previous developer left the code in a rough state: some of it is simply **buggy**, and some of it is **badly designed** — it works today but breaks the rules of good object-oriented design (for example, using the wrong access modifier for an attribute). Your job is to find both kinds of problems.

2. Learning objectives

This exercise gives you practice with the core OOP tools in TypeScript:

- **Access modifiers** — `public` / `protected` / `private`, and choosing the right one for each attribute.
- **Mutation modifier** — `readonly` for values that must never change after construction.
- **Constructors** — initializing state correctly, including calling `super(...)`.
- **Method overloading** — giving one method several call signatures.
- **Method overriding** — a subclass replacing a base-class method (and the pitfalls).
- **Accessors & mutators** — `get` / `set` to expose state safely with validation.
- **Basic error handling** — rejecting invalid operations by throwing.

3. The bank’s rules (the specification)

Every issue you look for is a place where the code disagrees with one of these rules. Read them carefully; they are your checklist.

- **R1:** An account’s **account number** is assigned once when the account is opened and must **never change** afterward. Any part of the program may read it.
- **R2:** An account’s **balance** may only change through `deposit`, `withdraw`, or `applyInterest`. No code outside the account classes may assign the balance directly. Subclasses may need to read and adjust it.
- **R3:** A **deposit or withdrawal amount must be positive**. A normal account can never be overdrawn — you cannot withdraw more than the current balance.
- **R4:** A **savings account’s interest rate** is a fraction (e.g. $0.02 = 2\%$). It must be between 0 and 0.25 inclusive; any attempt to set an invalid rate must be rejected.

- **R5:** A **checking account** may go negative, but never below `-overdraftLimit`. The overdraft limit is fixed when the account is opened.
- **R6:** A **transfer** moves money from one account to another. If the withdrawal is not allowed, the deposit must not happen — money can never be created out of thin air.

4. Getting started

```
npm install
npm run dev      # open the URL Vite prints
npm run build    # type-check (this reveals at least one problem)
```

Open the app and try every button. Notice what **looks wrong** (a balance that can't be right, an action that should have been blocked, a feature that errors out). Then read the code in `src/` and compare it against the rules above.

Suggested reading order: `account.ts` → `savingsAccount.ts` → `checkingAccount.ts` → `bank.ts`. Each file's comment header restates the rules it is supposed to follow.

5. Your tasks

Task 1 — Find and document the issues (write-up)

Find the problems in the code. There are **about seven**. For **each** one, write a short entry recording:

- **Location** — file and roughly where (method or attribute name).
- **Kind** — is it a **bug** (wrong behavior) or a **design flaw** (works, but violates a rule / good OOP)? Some are both.
- **Which rule / principle it breaks** — cite R1–R6 above, or the OOP concept (e.g. “encapsulation — wrong access modifier”).
- **The fix** — one sentence on what you changed.

You may keep your write-up in a separate document, or as comments in the code marked `// ISSUE:`. A blank table you can copy:

#	Location	Bug / Design	Rule broken + fix

Task 2 — Fix the issues in code

Correct every issue you find. When you are done:

- `npm run build` completes with **no errors**.
- In the app: the savings account shows its real opening balance; a normal account cannot be overdrawn; the checking account respects its overdraft limit; an out-of-range interest rate is rejected; deposits/withdrawals of zero or negative amounts are rejected.

Task 3 — Implement the transfer feature

The `Bank.transfer(fromNumber, toNumber, amount)` method in `bank.ts` currently throws “not implemented.” Implement it to satisfy **R6**:

- Look both accounts up with `getAccount(...)` (it already throws a helpful error for an unknown number — let that propagate).
- Withdraw from the source, then deposit into the destination, **reusing** the existing `deposit / withdraw` methods — do not touch balances directly.
- If the withdrawal throws (insufficient funds / overdraft exceeded), make sure the deposit never runs.

Verify in the app: transferring within the source’s available funds updates both accounts; a transfer that is too large is rejected and **neither** balance changes.

6. What to hand in

- **Add your name to the README file.**
- **Your Task 1 write-up (the table or code comments). Put that in a new section in the README file.**
- **Your fixed `src/` folder, with `npm run build` passing and transfer working. Commit and push your code.**

7. Hints

- Not every problem is caught by the compiler. `npm run build` finds one of them; the rest you find by **reading** and by **watching the app misbehave**.
- For each attribute, ask: “Who is allowed to read this? Who is allowed to change it?” That answers public vs protected vs private, and whether it should be `readonly` or hidden behind a `get/set`.
- If a subclass method is supposed to replace a base method but doesn’t seem to run, check the **exact spelling and signature** — and consider the `override` keyword.
- A method that needs two different call shapes wants **overload signatures**, not two implementations.